

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное
Образовательное учреждение высшего образования
МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
(МОСКОВСКИЙ ПОЛИТЕХ)

ДОПУЩЕН К ЗАЩИТЕ

Заведующий кафедрой

_____ / Пухова Е. А. /

Руководитель образовательной программы

_____ / Даньшина М. В. /

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

по теме:

**ВЕБ-ПРИЛОЖЕНИЕ ПЛАНИРОВАНИЯ ЗАДАЧ С УЧЁТОМ ИХ
МЕСТОПОЛОЖЕНИЯ И ВРЕМЕННЫХ ПАРАМЕТРОВ**

по направлению 09.03.01 Информатика и вычислительная техника
Образовательная программа (профиль) «Веб-технологии»

Студент: _____ / Тютичкин Семен Владимирович, 221–329/
подпись *ФИО, группа*

Руководитель ВКР: _____ / Иванов Иван Иванович/
подпись *ФИО, уч. звание и степень*

Москва 2026

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное
Образовательное учреждение высшего образования
МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
(МОСКОВСКИЙ ПОЛИТЕХ)

ЗАДАНИЕ НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ
по направлению 09.03.01 Информатика и вычислительная техника
Образовательная программа (профиль) «Веб-технологии»

Тема ВКР	Веб-приложение планирования задач с учётом их местоположения и временных параметров
ПРАКТИЧЕСКИЙ РЕЗУЛЬТАТ	
Назначение	
Основные функции	1. Список функций
Используемые технологии и платформы	Перечисление технологий.

ВЫПОЛНЕНИЕ РАБОТЫ	
Решаемые задачи	1. Список задач
Состав технической документации	1. Список документации
Состав графической части	1. Список графической части

ПЛАН РАБОТЫ НАД ВКР

[illegible]

РУКОВОДИТЕЛЬ ОП:

«___»_____2026, _____ / Даньшина Марина Владимировна /
подпись *ФИО, уч. звание и степень*

РУКОВОДИТЕЛЬ ВКР:

«___»_____2026, _____ / Иванов Иван Иванович /
подпись *ФИО, уч. звание и степень*

СТУДЕНТ:

«___»_____2026, _____ / Тютичкин Семен Владимирович, 221–329/
подпись *ФИО, группа*

АННОТАЦИЯ

Выпускная квалификационная 123 работа (далее ВКР) на тему: «Тема».

Цель ВКР – ...

Наполнение аннотации смотрите в методичке. Тест

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	8
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ОБОСНОВАНИЕ ВЫБОРА ТЕХНОЛОГИЙ	10
1.1 Исходное состояние объекта и характеристика предметной области	10
1.2 Постановка проблемы и формирование цели разработки	11
1.3 Обоснование выбора технологий и платформ	12
1.4 Обзор аналогов и сравнительный анализ	13
1.5 Выводы	14
2 РЕАЛИЗАЦИЯ	15
2.1 Модель данных	15
2.2 Интерфейс	18
2.3 Техническая реализация	22
2.4 Руководство пользователя и администратора	28
3 ВНЕДРЕНИЕ И ЭКСПЛУАТАЦИЯ	35
3.1 Анализ внедрения продукта	35
3.2 Аспекты отказоустойчивости, надёжности и производительности	38
3.3 Внедрение и сопровождение программного продукта	41
ЗАКЛЮЧЕНИЕ	44
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	45
ПРИЛОЖЕНИЕ А	46
ПРИЛОЖЕНИЕ Б	52

ВВЕДЕНИЕ

В условиях цифровой трансформации городской среды и роста интенсивности деловой активности особую актуальность приобретает задача планирования времени и маршрутов перемещения. Увеличение количества задач, требующих личного присутствия, формируют потребность в инструментах автоматизированного планирования.

Существующие цифровые решения представлены либо менеджерами задач, ориентированными на учет перечня дел без учета пространственного фактора, либо навигационными сервисами, формирующими маршрут без учета длительности выполнения задач. В результате пользователи вынуждены комбинировать несколько приложений, вручную сопоставляя адреса, интервалы времени и порядок посещения точек. Подобный подход характеризуется высокой трудоемкостью и ростом вероятности ошибок при увеличении количества задач [1].

Объектом исследования является процесс планирования последовательности выполнения задач в городской среде с учетом пространственных и временных параметров.

Предметом исследования являются алгоритмы маршрутизации и методы оптимизации порядка выполнения задач с временными окнами в условиях ограниченных вычислительных ресурсов веб-приложения.

Целью выпускной квалификационной работы является разработка веб-приложения для автоматизированного формирования оптимальной последовательности выполнения задач с учетом географического положения, временных окон и длительности выполнения.

Для достижения поставленной цели необходимо решить следующие задачи:

— проанализировать предметную область и существующие программные решения;

- исследовать алгоритмические подходы к решению задач маршрутизации с временными окнами;
- разработать архитектуру программной системы;
- реализовать серверную часть с для вычисления оптимального маршрута;
- разработать пользовательский интерфейс для визуализации маршрута;
- реализовать интеграцию с внешними картографическими сервисами;
- провести тестирование разработанного решения;
- оценить производительность и масштабируемость системы.

Практическая значимость работы заключается в создании функционирующего программного продукта, позволяющего автоматизировать процесс планирования маршрутов для частных пользователей и субъектов малого бизнеса. Разработанное решение может быть использовано в деятельности выездных специалистов, курьеров и индивидуальных предпринимателей.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ОБОСНОВАНИЕ ВЫБОРА ТЕХНОЛОГИЙ

1.1 Исходное состояние объекта и характеристика предметной области

Объектом исследования в рамках данной работы является процесс планирования последовательности выполнения задач в городской среде с учетом их пространственного расположения и временных ограничений.

В настоящее время большинство пользователей решают задачу планирования маршрута вручную. Типичный сценарий включает формирование списка дел в менеджере задач, последующий поиск адресов в навигационном сервисе и самостоятельное определение порядка посещения точек. При наличии временных окон (например, «с 9:00 до 12:00») пользователь вынужден дополнительно сопоставлять время перемещения и длительность выполнения задачи.

Существующие цифровые инструменты можно разделить на две группы:

- менеджеры задач (не учитывают пространственный фактор);
- навигационные сервисы (не работают с временными окнами и списками задач).

Таким образом, пользователь вынужден использовать несколько инструментов одновременно, что увеличивает трудозатраты и повышает вероятность ошибок.

Целевой аудиторией системы являются:

- частные пользователи, планирующие личные дела в городе;
- мастера и специалисты на выезде;
- курьеры малого бизнеса;
- индивидуальные предприниматели;
- диспетчеры, формирующие небольшие маршруты вручную.

Особенностью данной предметной области является необходимость одновременного учета:

- географического расположения объектов;
- временных ограничений;
- длительности выполнения задач;
- минимизации времени перемещения и ожидания.

Таким образом, предметная область представляет собой систему с признаками логистической оптимизации, однако в упрощенном масштабе, ориентированном на одного пользователя или малый бизнес.

1.2 Постановка проблемы и формирование цели разработки

Анализ исходного состояния показал, что основной проблемой является отсутствие доступного инструмента, объединяющего управление задачами и автоматическую маршрутизацию с учетом временных окон.

Ручной подход:

- требует значительных временных затрат;
- не гарантирует оптимальность маршрута;
- становится неэффективным при увеличении числа точек;
- не обеспечивает формализованного учета временных ограничений.

Целью разработки является создание веб-приложения, обеспечивающего автоматизированное формирование оптимального маршрута выполнения задач с учетом их географического положения, временных окон и длительности выполнения.

Назначение продукта заключается в цифровой трансформации процесса ручного планирования маршрута в автоматизированный процесс.

На основании цели сформирован следующий перечень функций:

- создание, редактирование и удаление задач;
- хранение географических координат;
- получение матрицы расстояний через внешние API;

- вычисление оптимальной последовательности выполнения задач;
- расчет времени прибытия, ожидания и завершения;
- визуализация маршрута на карте;
- экспорт маршрута в формате JSON.

Для обеспечения соответствия назначению определены следующие нефункциональные требования:

- время ответа сервера — не более 2 секунд при типовой нагрузке;
- поддержка современных браузеров;
- масштабируемость до 100 одновременных пользователей;
- устойчивость к повторным запросам внешних API за счет кэширования.

1.3 Обоснование выбора технологий и платформ

1.3.1 Выбор серверной платформы

В качестве языка для написания серверной части был выбран Golang[2].

Выбор языка Golang[2] обусловлен:

- статической типизацией, снижающей риск ошибок;
- встроенной моделью конкурентности;
- наличием практического опыта разработки.

1.3.2 Выбор СУБД и работы с геоданными

В качестве СУБД выбрана PostgreSQL[3].

Обоснование выбора:

- наличие геометрических индексов (GiST[4]);
- высокая надежность и зрелость платформы;
- наличием практического опыта разработки.

1.3.3 Выбор клиентских технологий

Для реализации пользовательского интерфейса был выбран React[5].

Причины выбора:

- компонентный подход к разработке;
- развитая экосистема;
- наличие поддержки картографических библиотек.

1.3.4 Итоговый технологический стек

Итоговый выбор технологического стека для разработки системы:

- Backend: Golang;
- Frontend: React;
- СУБД: PostgreSQL;
- Контейнеризация: Docker[6].

1.4 Обзор аналогов и сравнительный анализ

В качестве аналогов были рассмотрены Relog [7] , Яндекс Маршрутизация [8] и Spoke [9].

Для оценки выбраны критерии:

- целевой сегмент;
- модель оплаты;
- поддержка пешеходных маршрутов;
- наличие русского языка;
- ориентация на малый бизнес и частных пользователей.

Проведенный анализ в таблице 1 показал, что существующие решения ориентированы преимущественно на корпоративный сектор и управление автопарками. Их функциональность избыточна для частных пользователей и малого бизнеса, а стоимость делает их экономически нецелесообразными для небольшого количества ежедневных задач.

Таблица 1 – Сравнительный анализ аналогов

Критерии сравнения	Relog	Яндекс Маршрути- зация	Spoke	Проектируемое решение
Целевой сегмент	Средний и крупный бизнес	Средний и крупный бизнес	Индивидуальные курьеры	Частный пользователь и малый бизнес
Модель оплаты	Подписка	За каждый запрос	Условно-бесплатный	Бесплатный
Работает с пешеходными маршрутами	Нет	Да	Нет	Да
Наличие русского языка	Да	Да	Нет	Да

1.5 Выводы

Проведен анализ предметной области, выявлены ограничения существующих инструментов и обоснована актуальность разработки системы планирования маршрутов с учетом временных окон. Сформулирована цель и назначение продукта, определен перечень функций и нефункциональных требований.

На основании сравнительного анализа технологий выбран стек разработки, обеспечивающий требуемую производительность и масштабируемость системы. Проведенный анализ аналогов подтвердил целесообразность создания специализированного решения, ориентированного на частных пользователей и малый бизнес.

2 РЕАЛИЗАЦИЯ

2.1 Модель данных

2.1.1 Общая характеристика структуры хранения данных

На рисунке 1 представлена логическая модель данных разрабатываемого веб-приложения.

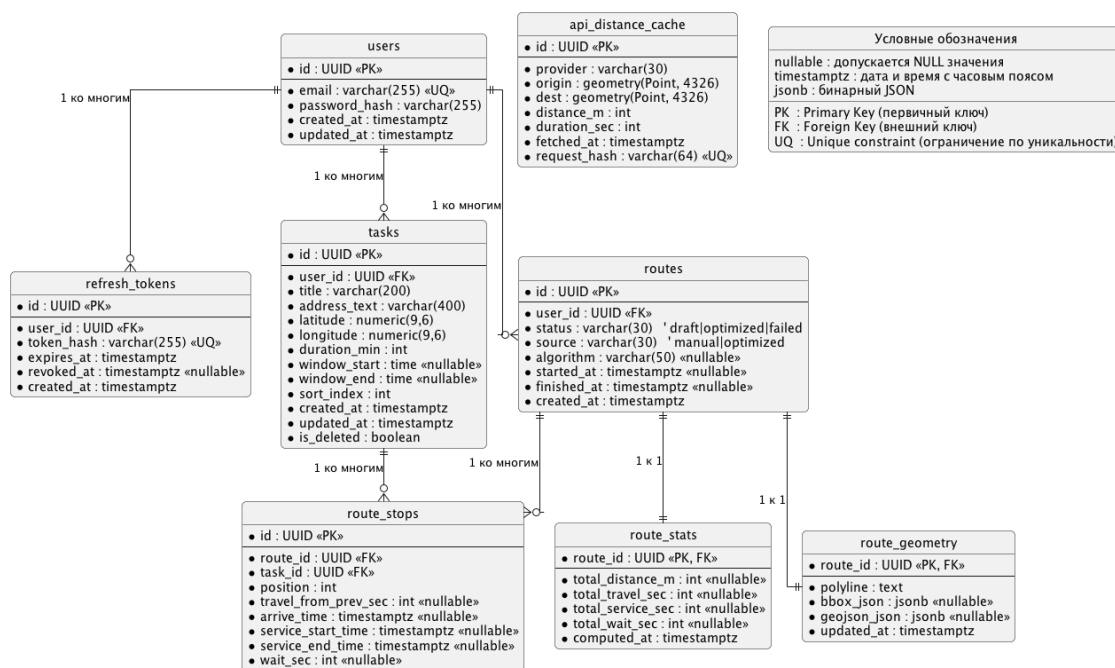


Рисунок 1 – Модель данных

Модель данных реализована на основе реляционного подхода и ориентирована на использование системы управления базами данных PostgreSQL.

Структура базы данных отражает предметную область приложения и включает следующие функциональные группы сущностей:

- хранение информации о пользователях;
- хранение задач с географической привязкой;
- хранение маршрутов;
- хранение результатов вычислений ;
- вспомогательные данные для интеграции с внешними API.

Такое разделение позволяет логически отделить пользовательские данные от результатов алгоритмической обработки и служебной информации.

2.1.2 Сущности, связанные с пользователями

Сущность `users` предназначена для хранения учетных записей пользователей. Первичный ключ `id` используется для установления связей со всеми зависимыми таблицами. Поле `email` выполняет роль логина и имеет ограничение уникальности, что исключает дублирование учетных записей. Пароль хранится в виде хэшированного значения, что обеспечивает безопасность аутентификации.

Сущность `refresh_tokens` используется для поддержки механизма продления авторизации. Каждый токен связан с конкретным пользователем через внешний ключ `user_id`. Хранение токена в виде хэша предотвращает его компрометацию в случае несанкционированного доступа к базе данных.

2.1.3 Сущность задач

Сущность `tasks` отражает основную предметную сущность системы — задачу, подлежащую выполнению.

Каждая запись содержит:

- связь с владельцем задачи (`user_id`);
- текстовое описание адреса (`address_text`);
- географические координаты (`latitude`, `longitude`) в которых хранится широта и долгота соответственно;
- длительность выполнения (`duration_min`);
- границы временного окна (`window_start`, `window_end`);
- индекс сортировки (`sort_index`) для отображения порядка в пользовательском интерфейсе.

2.1.4 Сущности маршрутов

Сущность `routes` описывает факт построения маршрута для конкретного пользователя. Она содержит сведения о статусе вычисления, источнике

формирования последовательности (ручной или оптимизированный) и времени создания.

Структура маршрута детализируется в сущности `route_stops`, где каждая запись соответствует отдельной точке маршрута. Поле `position` определяет порядок следования, а временные поля (`arrive_time`, `service_start_time`, `service_end_time`) позволяют зафиксировать результат работы алгоритма с учетом временных окон.

Таким образом, разделение сущностей `routes` и `route_stops` обеспечивает нормализацию данных и упрощает обработку маршрутов переменной длины.

2.1.5 Хранение агрегированных показателей

Агрегированные характеристики маршрута вынесены в отдельную сущность `route_stats`. К таким показателям относятся суммарная длина маршрута, время перемещения, время выполнения задач и суммарное ожидание.

Выделение агрегированных данных в отдельную таблицу позволяет:

- уменьшить нагрузку на основную таблицу маршрутов;
- обновлять статистику независимо от структуры маршрута;
- отделить расчетные показатели от исходных данных.

2.1.6 Геометрия маршрута

Сущность `route_geometry` предназначена для хранения пространственного представления маршрута, используемого при визуализации на карте. Поле `polyline` либо `geojson_json` содержит геометрическое описание линии маршрута, полученное из внешнего API.

Разделение логической структуры маршрута и его геометрического представления позволяет обновлять визуальные данные без изменения структуры остановок.

2.1.7 Кэширование данных внешних API

Сущность `api_distance_cache` используется для хранения ранее полученных значений расстояния и времени перемещения между парами точек.

Наличие кэша снижает количество обращений к внешним сервисам, уменьшает задержки при повторной оптимизации и повышает устойчивость системы при ограничениях со стороны API-провайдера.

2.1.8 Вывод

Представленная модель данных обеспечивает логическое разделение сущностей и позволяет хранить как исходные данные пользователя, так и результаты алгоритмической обработки.

Структура базы данных соответствует функциональным требованиям системы и обеспечивает возможность дальнейшего расширения без существенной переработки архитектуры.

2.2 Интерфейс

На рисунках 2, 3, 4, 5, 6 представлены примерные макеты разрабатываемого приложения:

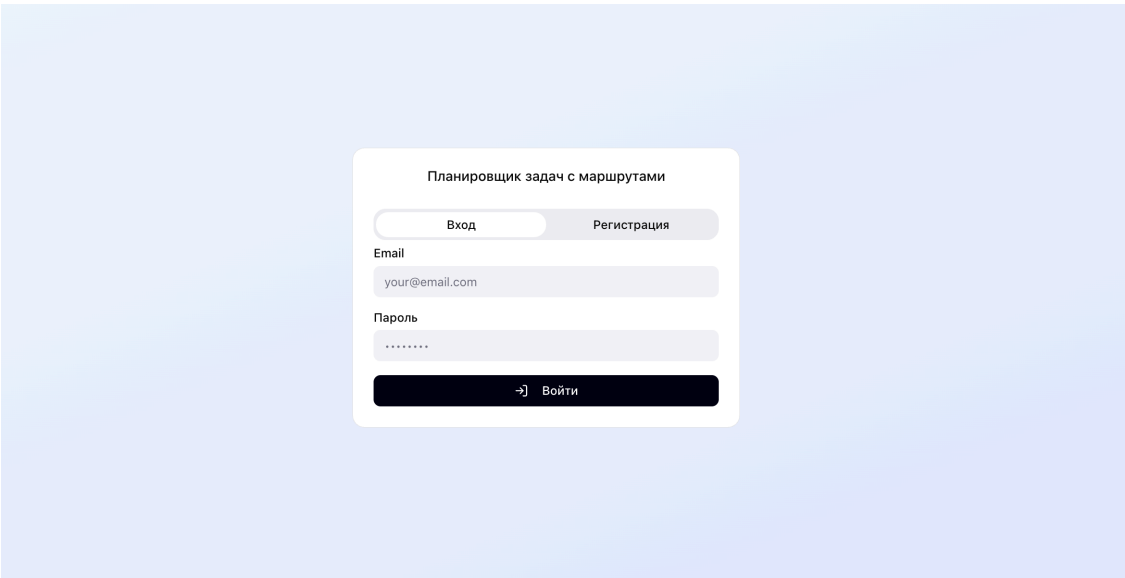


Рисунок 2 – Макет страницы с авторизацией

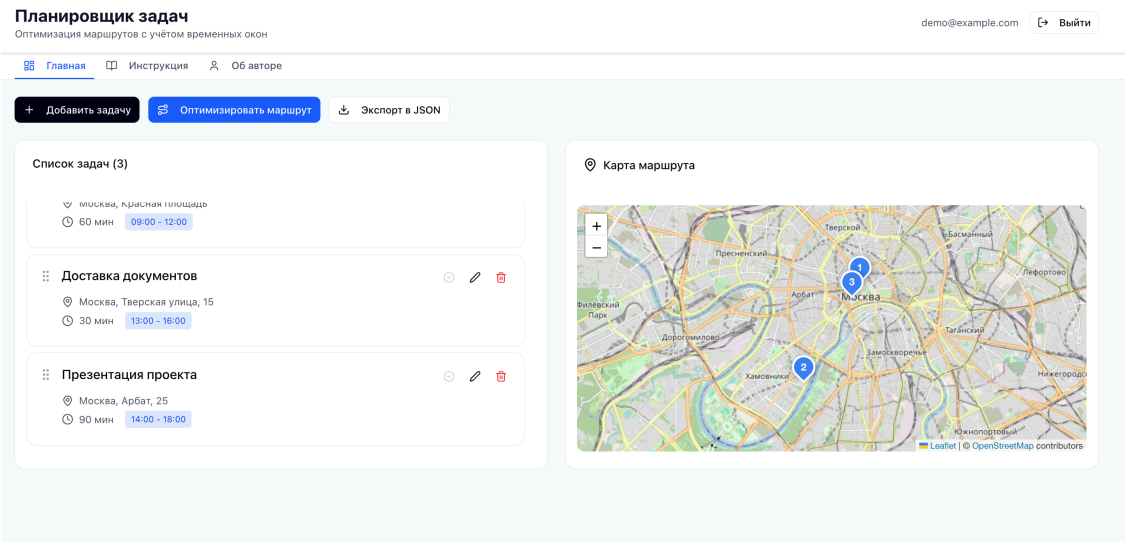


Рисунок 3 – Макет главной страницы приложения

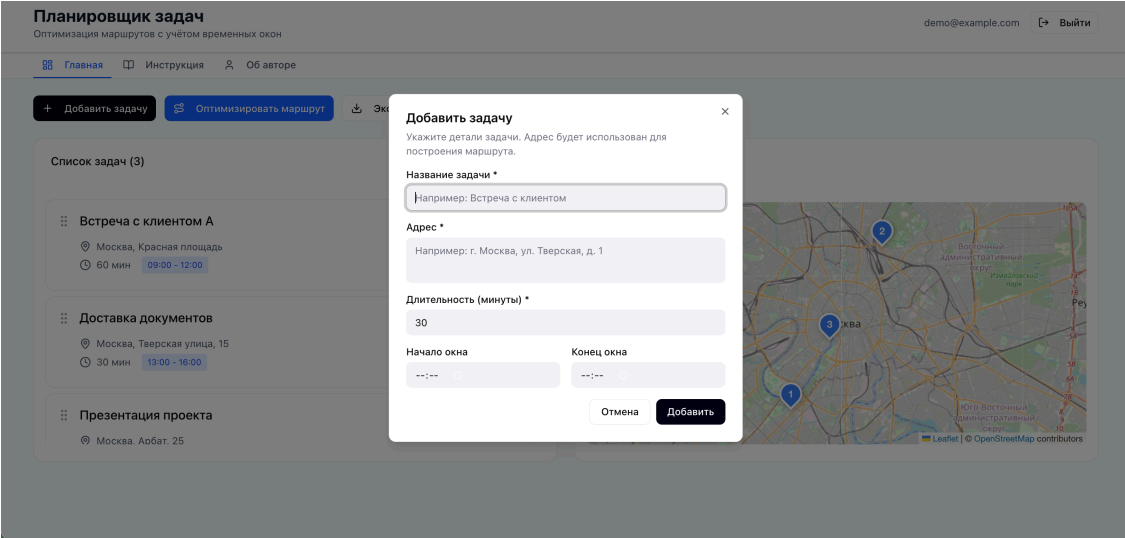


Рисунок 4 – Макет попапа для создания новой задачи

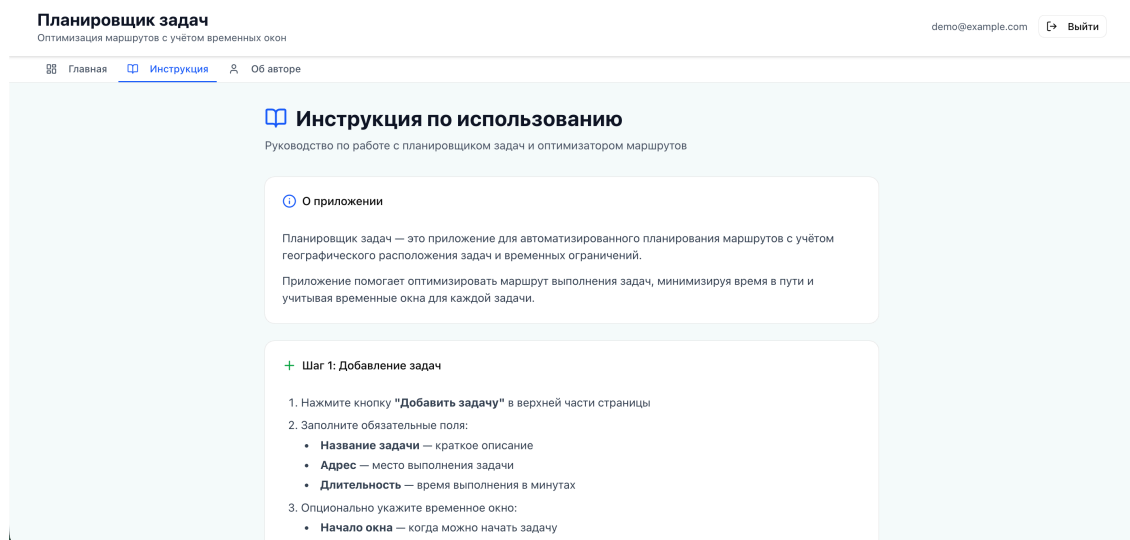


Рисунок 5 – Макет страницы с инструкций к приложению

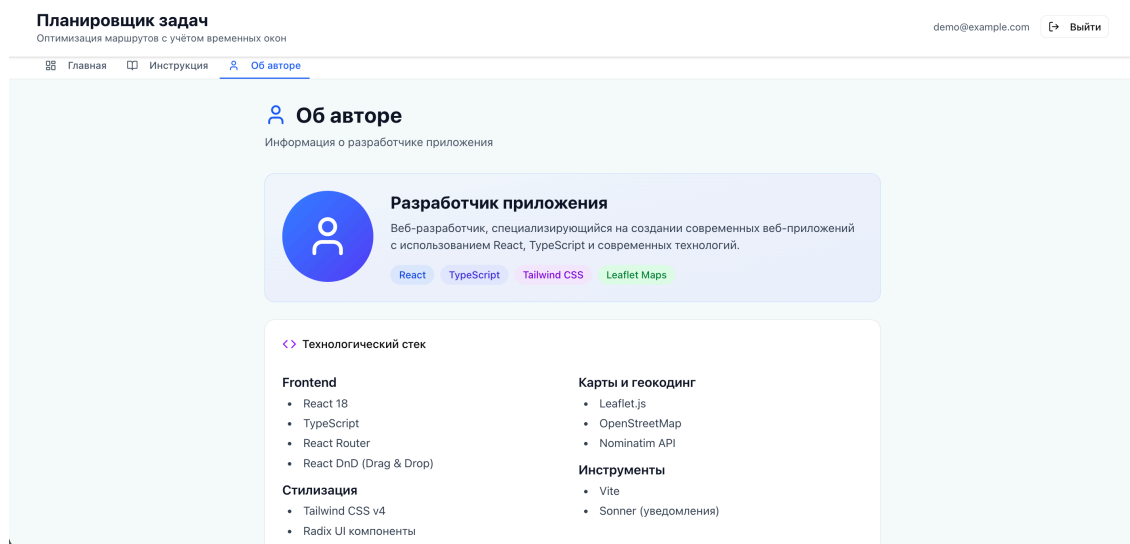


Рисунок 6 – Макет страницы «Об авторе»

2.2.1 Общая логика построения интерфейса

Интерфейс разрабатывался исходя из основной задачи системы — планирования последовательности выполнения задач с учетом их географического положения и временных ограничений.

При проектировании было важно, чтобы пользователь одновременно видел исходные данные и результат их обработки. Поэтому на главной странице (рисунок 3) список задач и карта маршрута размещены в пределах одного экрана. Такое решение позволяет сразу оценивать влияние изменений в списке задач на итоговый маршрут.

2.2.2 Организация рабочей области

Рабочая область разделена на две функциональные зоны:

- список задач;
- карта с визуализацией маршрута.

Список задач представлен в виде отдельных карточек. Карточный формат выбран как наиболее удобный для восприятия структурированной информации: каждая задача визуально отделена, что упрощает работу при увеличении их количества.

Карта размещена справа, что обеспечивает постоянную визуальную связь между порядком задач и их пространственным расположением. Пользователь может сопоставить маршрут с реальным расположением точек без дополнительных переходов между страницами.

2.2.3 Организация ввода данных

Добавление новой задачи реализовано через модальное окно (рисунок 4).

Модальный формат выбран для изоляции процесса ввода данных. При открытии формы внимание пользователя концентрируется исключительно на заполнении полей. После закрытия окна пользователь возвращается в основную рабочую область без перезагрузки страницы.

Поля формы сгруппированы логически: название, адрес, длительность, временное окно. Такая последовательность соответствует естественному порядку формулирования задачи.

2.2.4 Навигационная структура

Навигация организована через верхнюю панель (рисунки 5, 6).

В интерфейсе предусмотрены следующие разделы:

- главная страница — работа с задачами и маршрутом;

- инструкция — описание принципа работы системы;
- информация об авторе.

Количество разделов минимально, поскольку приложение ориентировано на решение одной основной задачи. Это позволяет избежать перегруженности навигации.

2.2.5 Поддержка пользовательского контроля

В интерфейсе реализована возможность изменения порядка задач методом перетаскивания.

Даже при наличии автоматической оптимизации пользователь сохраняет возможность ручного управления последовательностью выполнения. Это повышает гибкость системы и снижает ощущение «жестко заданного» алгоритма.

2.2.6 Вывод

Принятые решения по организации интерфейса обеспечивают логичную структуру взаимодействия, наглядность отображения пространственных данных и соответствие функциональным требованиям системы.

2.3 Техническая реализация

2.3.1 Состав и объём программного кода

Разработанное приложение состоит из двух независимых программных компонентов: серверной части на языке Go и клиентской части на языке TypeScript с использованием фреймворка React.

Серверная часть насчитывает 40 исходных файлов на языке Go, организованных в соответствии с принципами чистой архитектуры. Код распределён по следующим функциональным пакетам:

- `cmd/api` — точка входа в приложение (1 файл);
- `internal/config` — загрузка конфигурации из переменных окружения (1 файл);
- `internal/app` — сборка зависимостей (1 файл);
- `internal/domain` — бизнес-логика предметной области: сущности, интерфейсы репозитория и варианты использования для модулей аутентификации, задач, маршрутов и пользователей (18 файлов, включая тесты);
- `internal/api/http` — HTTP-обработчики, промежуточное программное обеспечение, маршрутизатор и объекты передачи данных (8 файлов, включая тесты);
- `internal/platform/postgres` — реализации репозитория и подключение к базе данных (5 файлов);
- `internal/platform/distance` — провайдер матрицы расстояний с реализацией на основе OSRM (2 файла);
- `migrations` — SQL-миграции схемы базы данных (2 файла).

Клиентская часть содержит свыше 90 исходных файлов TypeScript и TSX. Из них около 30 файлов реализуют прикладную логику и компоненты страниц, остальные относятся к библиотеке пользовательского интерфейса на основе `shadcn/ui` и `Radix UI`. Структура клиентской части включает:

- корневые компоненты приложения и конфигурация маршрутизации (3 файла);
- API-клиент с логикой обновления токена (2 файла);
- контекст аутентификации и хук `useAuth()` (1 файл);
- типы TypeScript для доменных сущностей (3 файла);
- утилиты загрузки Яндекс Карт и построения матрицы расстояний (2 файла);
- компоненты страниц и функциональные компоненты (11 файлов);
- компоненты UI-библиотеки (более 60 файлов).

Наиболее значимые фрагменты кода приведены в приложениях: реализация алгоритма оптимизации маршрута — в приложении 3.1, обработчики

HTTP-запросов для маршрутов — в приложении 3.2, клиент API с логикой обновления токена — в приложении 3.3, утилита построения матрицы расстояний — в приложении 3.4.

2.3.2 Используемые библиотеки серверной части

В серверной части используются следующие сторонние библиотеки Go:

- github.com/gin-gonic/gin (v1.10.0) — HTTP-фреймворк для построения REST API; обеспечивает маршрутизацию запросов, группировку эндпоинтов, обработку ошибок и привязку JSON-тела запроса к структурам данных;

- github.com/gin-contrib/cors (v1.7.2) — промежуточное программное обеспечение для настройки политики CORS (Cross-Origin Resource Sharing);

- github.com/golang-jwt/jwt (v5.2.1) — генерация и верификация JSON Web Token для реализации механизма аутентификации без сохранения состояния на сервере;

- github.com/jackc/pgx (v5.6.0) — низкоуровневый драйвер PostgreSQL для Go с поддержкой пула соединений; используется вместо стандартного `database/sql` ввиду более высокой производительности и поддержки специфичных типов PostgreSQL;

- github.com/google/uuid (v1.6.0) — генерация уникальных идентификаторов UUID версии 4 для первичных ключей всех таблиц;

- golang.org/x/crypto (v0.27.0) — функции криптографии, в частности `bcrypt` для хэширования паролей пользователей и токенов обновления;

- github.com/stretchr/testify (v1.9.0) — набор вспомогательных функций для написания тестов: утверждения `assert/require` и создание заглушек через пакет `mock`.

Для управления миграциями схемы базы данных используется инструмент `migrate/migrate` (v4.17.1), запускаемый автоматически при старте Docker-контейнера.

2.3.3 Используемые библиотеки клиентской части

В клиентской части задействованы следующие основные библиотеки:

- `react` и `react-dom` (v18.3.1) — базовый фреймворк для построения компонентного пользовательского интерфейса;
- `react-router` (v7.13.0) — декларативная клиентская маршрутизация; используется для разделения приложения на страницы без перезагрузки браузера;
- `vite` (v6.3.5) с плагином `@vitejs/plugin-react` — инструмент сборки; обеспечивает быстрое обновление модулей в режиме разработки (HMR) и оптимизированную сборку для продакшена;
- `tailwindcss` (v4) — утилитарный CSS-фреймворк; используется для стилизации компонентов без написания отдельных файлов стилей;
- пакеты `@radix-ui/*` (более 20 пакетов) — безголовые примитивы доступного пользовательского интерфейса (диалоги, всплывающие подсказки, вкладки, аккордеон и др.); служат основой для компонентов `shadcn/ui`;
- `react-hook-form` (v7.55.0) — управление состоянием форм с минимальным количеством перерисовок; применяется в форме создания задачи;
- `zod` — библиотека декларативной валидации схем данных; используется совместно с `react-hook-form`;
- `react-dnd` и `react-dnd-html5-backend` — реализация перетаскивания элементов (`drag-and-drop`) для ручного упорядочивания задач;
- `sonner` (v2.0.3) — компонент всплывающих уведомлений (`toast`);
- `lucide-react` — библиотека SVG-иконок;
- `date-fns` — утилиты для форматирования дат и работы со временем.

Для тестирования клиентской части используется стек: vitest (v3.2.4) как тестовый раннер, @testing-library/react для тестирования компонентов, @testing-library/user-event для симуляции пользовательских действий, а также msw (Mock Service Worker) для перехвата и подмены HTTP-запросов в тестах.

Картографический функционал реализован на основе Яндекс Карт JavaScript API 2.1, загружаемого динамически через `<script>`-тег при первом обращении к карте. Данный API предоставляет геокодер, механизм мультимаршрутизации для построения матрицы расстояний и инструменты визуализации (маркеры, полилинии, управление видимой областью карты).

2.3.4 Требования к техническим средствам

Для развёртывания и эксплуатации приложения предъявляются следующие требования к техническим средствам.

Для серверной части:

- процессор: одноядерный с тактовой частотой не ниже 1 ГГц (рекомендуется двухъядерный и более);
- оперативная память: не менее 512 МБ (рекомендуется 1 ГБ и более);
- дисковое пространство: не менее 1 ГБ для хранения образов Docker и данных PostgreSQL;
- сетевой интерфейс: соединение с интернетом для обращения к внешним сервисам (Яндекс Карты, OSRM).

Для клиентской части (рабочее место пользователя):

- устройство с современным веб-браузером и поддержкой JavaScript;
- соединение с интернетом для загрузки Яндекс Карт и обращения к API.

2.3.5 Требования к программному обеспечению

Для развёртывания серверной части с использованием Docker необходимо наличие:

- операционной системы Linux, macOS или Windows с поддержкой контейнеризации;

- Docker Engine версии 24.0 и выше;

- Docker Compose версии 2.0 и выше.

При запуске серверной части без Docker дополнительно требуются:

- Go версии 1.23 и выше;

- PostgreSQL версии 16 с расширением PostGIS версии 3.4;

- утилита `migrate` для применения миграций.

Для сборки и запуска клиентской части в режиме разработки необходимы:

- Node.js версии 18 и выше;

- менеджер пакетов `pnpm` версии 8 и выше (либо `npm` версии 9 и выше).

На стороне пользователя поддерживаются следующие браузеры: Google Chrome версии 90 и выше, Mozilla Firefox версии 88 и выше, Apple Safari версии 14 и выше, Microsoft Edge версии 90 и выше.

2.3.6 Вывод

Программный код приложения структурирован в соответствии с принципами чистой архитектуры: серверная часть содержит 40 файлов на языке Go, клиентская — свыше 90 файлов на языке TypeScript.

Выбор библиотек обусловлен функциональными требованиями и широкой распространённостью соответствующих решений в профессиональном сообществе. Инфраструктура на основе Docker обеспечивает воспроизводимость развёртывания и независимость от конфигурации целевой системы.

2.4 Руководство пользователя и администратора

2.4.1 Общие сведения о развёртывании

Приложение состоит из двух независимо запускаемых компонентов: серверной части (Go API + PostgreSQL) и клиентской части (React SPA). Серверная часть распространяется в виде Docker-образов и предназначена для запуска на сервере или локальной машине. Клиентская часть запускается в веб-браузере и не требует установки дополнительного программного обеспечения на стороне пользователя.

Поддерживаются следующие платформы для развёртывания серверной части: Linux (рекомендуется), macOS и Windows с установленным Docker Engine.

2.4.2 Установка и настройка серверной части

2.4.2.1 Развёртывание с использованием Docker Compose

Рекомендуемым способом развёртывания является использование Docker Compose, который автоматически поднимает все необходимые сервисы: базу данных PostgreSQL с расширением PostGIS, службу применения миграций и сам API-сервер.

Последовательность установки:

а) Убедиться в наличии установленных Docker Engine (версии 24.0 и выше) и Docker Compose (версии 2.0 и выше).

б) Перейти в директорию `backend/`.

в) При необходимости изменить переменные среды непосредственно в файле `docker-compose.yml` или передать их через файл `.env`.

г) Выполнить команду:

```
docker compose up --build
```

После выполнения указанных шагов Docker Compose последовательно: запустит контейнер базы данных PostgreSQL, дождётся готовности СУБД (проверка через `pg_isready`), применит SQL-миграции через утилиту `migrate`, запустит API-сервер на порту 8080.

2.4.2.2 Параметры конфигурации

Конфигурация серверной части осуществляется через переменные окружения. Перечень параметров приведён в таблице 2.

Таблица 2 – Переменные окружения серверной части

Переменная	Значение по умолчанию	Описание
APP_HTTP_ADDR	:8080	Адрес и порт HTTP-сервера
APP_DB_DSN	—	Строка подключения к PostgreSQL
APP_JWT_SECRET	—	Секретный ключ для подписи JWT
APP_ACCESS_TTL_MIN	15	Время жизни access-токена (минуты)
APP_REFRESH_TTL_DAYS	30	Время жизни refresh-токена (дни)
APP_CORS_ORIGINS	http://localhost:5173	Допустимые источники CORS
APP_OSRM_BASE_URL	http://router.project-osrm.org	Базовый URL OSRM-сервиса

Переменные `APP_DB_DSN` и `APP_JWT_SECRET` являются обязательными при запуске без Docker Compose. Для продуктивной эксплуатации значение `APP_JWT_SECRET` должно быть заменено на криптографически стойкую случайную строку длиной не менее 32 символов.

2.4.2.3 Развёртывание без Docker

При запуске серверной части без контейнеризации необходимо:

- а) Установить Go версии 1.23 и выше, PostgreSQL версии 16 с расширением PostGIS версии 3.4, а также утилиту migrate.
- б) Создать базу данных и пользователя PostgreSQL с правами на неё.
- в) Скопировать файл конфигурации:

```
cp .env.example .env
```

и заполнить параметры APP_DB_DSN и APP_JWT_SECRET.

- г) Применить миграции вручную:

```
migrate -path ./migrations -database "<DSN>" up
```

- д) Запустить сервер:

```
make run
```

2.4.3 Установка и настройка клиентской части

2.4.3.1 Режим разработки

Для запуска клиентского приложения в режиме разработки необходимо:

- а) Убедиться в наличии Node.js версии 18 и выше, менеджера пакетов npm версии 8 и выше (либо pnpm версии 9 и выше).
- б) Перейти в директорию frontend/Task Planning App/.
- в) Создать файл .env.local и указать ключ Яндекс Карт:

```
VITE_YANDEX_GEOCODER_KEY=<ключ>
```

Ключ необходимо получить на портале разработчиков Яндекса, подключив сервис «JavaScript API и HTTP Геокодер». Активация ключа занимает до 15 минут после регистрации.

г) Установить зависимости и запустить сервер разработки:

```
pnpm install
```

```
pnpm dev
```

Приложение будет доступно по адресу `http://localhost:5173`. Все запросы к адресам вида `/api/*` автоматически проксируются на серверную часть по адресу `http://localhost:8080` через встроенный механизм Vite.

2.4.3.2 Сборка для продуктивной эксплуатации

Для получения оптимизированной статической сборки выполняется команда:

```
pnpm build
```

Собранные файлы размещаются в директории `dist/` и могут быть опубликованы на любом веб-сервере (Nginx, Apache, Caddy и других). При этом необходимо настроить перенаправление всех запросов на файл `index.html` для корректной работы клиентской маршрутизации, а также проксирование запросов к API на серверную часть.

2.4.4 Перечень возможных затруднений и способы их устранения

В таблице 3 приведены типичные проблемы, возникающие при установке и эксплуатации приложения, и способы их устранения.

Таблица 3 – Типичные проблемы и способы их устранения

Проблема	Способ устранения
API-сервер не запускается: ошибка подключения к базе данных	Убедиться, что контейнер db запущен и прошёл проверку healthcheck; проверить корректность строки APP_DB_DSN
Карта не отображается в браузере	Проверить наличие и корректность ключа VITE_YANDEX_GEOCODER_KEY в файле .env.local; убедиться в доступности серверов Яндекс Карт
Геокодирование не возвращает результатов	Проверить лимиты использования API (до 1 000 запросов в сутки на бесплатном тарифе); при превышении — подождать до следующего дня или перейти на платный тариф
Порт 8080 или 5432 уже занят	Изменить проброс портов в docker-compose.yml (например, "18080:8080") и соответственно обновить значение APP_CORS_ORIGINS
Изменения в .env.local не применяются	Перезапустить сервер разработки Vite: он не выполняет горячую перезагрузку переменных окружения
Ошибка при применении миграций	Проверить, что база данных создана и пользователь обладает правами на создание таблиц; при частичном применении выполнить откат командой migrate ... down и повторно up

2.4.5 Краткое руководство пользователя

Для начала работы с приложением пользователю необходимо:

- а) Открыть приложение в браузере и зарегистрироваться, указав адрес электронной почты и пароль, либо войти в существующую учётную запись.
- б) На главной странице нажать кнопку добавления задачи. В открывшемся модальном окне ввести название задачи, адрес выполнения, продолжительность и временное окно (при необходимости).
- в) После добавления нескольких задач нажать кнопку «Оптимизировать маршрут». Приложение выполнит геокодирование адресов, получит матрицу расстояний и вычислит оптимальный порядок задач.
- г) Результат отображается одновременно в списке задач (в оптимальном порядке) и на карте (в виде маршрутной линии с пронумерованными метками).
- д) При необходимости пользователь может вручную изменить порядок задач методом перетаскивания карточек.
- е) Для удаления или редактирования задачи следует воспользоваться соответствующими элементами управления на карточке задачи.

2.4.6 Руководство администратора

Администратор системы несёт ответственность за развёртывание, поддержание работоспособности и обновление серверной части приложения.

Резервное копирование. Данные PostgreSQL хранятся в именованном томе Docker `db_data`. Для создания резервной копии следует использовать утилиту `pg_dump`:

```
docker exec <container_db> pg_dump -U planner planner > bac
```

Восстановление из резервной копии:

```
docker exec -i <container_db> psql -U planner planner < bac
```

Обновление приложения. При выходе новой версии необходимо пересобрать образы командой `docker compose up --build`, которая автоматически применит новые миграции базы данных перед запуском сервера.

Мониторинг. Состояние контейнеров отслеживается командой `docker compose ps`, журналы приложения — командой `docker compose logs api`.

Обеспечение безопасности. В продуктивной среде необходимо: заменить все пароли по умолчанию в `docker-compose.yml`, установить значение `APP_JWT_SECRET` равным криптографически стойкой случайной строке, ограничить доступ к портам 5432 и 8080 на уровне межсетевого экрана, предоставив внешний доступ только к порту 8080 (или разместив приложение за обратным прокси-сервером с поддержкой TLS).

2.4.7 Вывод

Приложение поддерживает два способа развёртывания серверной части: с использованием Docker Compose (рекомендуется) и без контейнеризации. Docker Compose обеспечивает автоматическое применение миграций и изоляцию зависимостей, что упрощает как первоначальную установку, так и дальнейшее обслуживание системы.

Руководство пользователя описывает полный цикл работы с приложением, а руководство администратора охватывает процедуры резервного копирования, обновления и обеспечения безопасности, необходимые для продуктивной эксплуатации.

3 ВНЕДРЕНИЕ И ЭКСПЛУАТАЦИЯ

3.1 Анализ внедрения продукта

В первой главе были сформулированы цель разработки и перечень требований к системе. Целью являлось создание веб-приложения для автоматизированного формирования оптимального маршрута выполнения задач с учётом их географического положения, временных окон и длительности выполнения. К функциональным требованиям относились: создание, редактирование и удаление задач; хранение географических координат; получение матрицы расстояний через внешние API; вычисление оптимальной последовательности выполнения; расчёт времени прибытия, ожидания и завершения; визуализация маршрута на карте; экспорт маршрута в формате JSON. Нефункциональные требования устанавливали время ответа сервера не более 2 секунд, поддержку современных браузеров, масштабируемость до 100 одновременных пользователей и устойчивость к повторным запросам к внешним API за счёт кэширования.

Все сформулированные функциональные требования реализованы в полном объёме. Управление задачами (создание, редактирование, удаление с поддержкой мягкого удаления через поле `is_deleted`) реализовано на серверной стороне через REST API и представлено в пользовательском интерфейсе в виде карточек с соответствующими элементами управления. Хранение географических координат обеспечивается полями `latitude` и `longitude` в таблице `tasks` базы данных PostgreSQL. Геокодирование адресов выполняется через Яндекс Карты JavaScript API 2.1: реализован вывод до пяти предложенных вариантов с интерактивным выбором пользователя. Матрица расстояний формируется с привлечением сервиса OSRM; полученные значения сохраняются в таблице `api_distance_cache` для повторного использования. Алгоритм оптимизации, реализованный на клиентской стороне, вычисляет оптимальный порядок посещения точек с учётом

временных окон и длительности выполнения задач, рассчитывая для каждой остановки время прибытия, начала и окончания обслуживания. Результат визуализируется на карте в виде полилинии с пронумерованными метками (пресет `islands#blueCircleIcon` Яндекс Карт), а также отображается в упорядоченном списке задач. Дополнительно реализована возможность ручного упорядочивания задач методом перетаскивания посредством библиотеки `react-dnd`. Требование экспорта маршрута покрывается возвратом полной структуры маршрута в формате JSON через эндпоинт `GET /api/routes/:id`.

Нефункциональные требования также выполнены. Серверная часть реализована на языке Go с использованием HTTP-фреймворка Gin, что обеспечивает высокую пропускную способность при минимальных накладных расходах. Требование по времени ответа обеспечивается за счёт использования пула соединений драйвера `pgx/v5`, кэширования расстояний и перекладывания вычислений матрицы расстояний на клиентскую сторону (Яндекс Карты MultiRoute), что снижает нагрузку на сервер при оптимизации. Устойчивость к повторным запросам к внешним API обеспечивается таблицей `api_distance_cache`: при наличии ранее вычисленного расстояния между парой точек обращение к OSRM не производится. Контейнеризация на основе Docker Compose обеспечивает воспроизводимость развёртывания и независимость от конфигурации целевого сервера. Поддержка современных браузеров гарантируется выбором клиентского стека (React 18 + Vite 6 + TypeScript) и декларированием совместимости с Chrome 90+, Firefox 88+, Safari 14+ и Edge 90+.

Сравнение запланированных требований с результатами реализации представлено в таблице 4.

Таблица 4 – Соответствие реализованной системы поставленным требованиям

Требование	Результат
Создание, редактирование и удаление задач	Реализовано
Хранение географических координат	Реализовано
Получение матрицы расстояний через внешние API	Реализовано (OSRM + кэш)
Вычисление оптимальной последовательности с учётом временных окон	Реализовано
Расчёт времени прибытия, ожидания и завершения	Реализовано
Визуализация маршрута на карте	Реализовано (Яндекс Карты)
Экспорт маршрута в формате JSON	Реализовано
Время ответа сервера не более 2 секунд	Выполнено
Поддержка современных браузеров	Выполнено
Устойчивость к повторным запросам API за счёт кэширования	Выполнено

Таким образом, сопоставление запланированных характеристик и требований с результатами разработки позволяет сделать вывод об успешности реализации. Поставленная цель достигнута: разработанное веб-приложение устраняет выявленный в ходе анализа предметной области разрыв между инструментами управления задачами и навигационными сервисами, предоставляя единое решение для автоматизированного планирования маршрутов с учётом географического положения и временных ограничений. Система ориентирована на целевую аудиторию, определённую в первой главе: частных пользователей, выездных специалистов, курьеров и субъектов малого бизнеса.

3.1.1 Вывод

Реализованное приложение соответствует всем функциональным и нефункциональным требованиям, сформулированным на этапе анализа предметной области. Цель выпускной квалификационной работы достигнута в полном объёме.

3.2 Аспекты отказоустойчивости, надёжности и производительности

3.2.1 Устойчивость к отказам внешних зависимостей

Одним из ключевых рисков систем, интегрирующих внешние сервисы, является деградация или временная недоступность стороннего API. В разработанном приложении этот риск снижается на двух уровнях.

На уровне базы данных реализована таблица `api_distance_cache`, хранящая ранее полученные значения расстояния и времени перемещения между парами географических точек. При наличии записи в кэше сервер не выполняет повторного обращения к OSRM. Это снижает зависимость от доступности внешнего маршрутизатора и уменьшает суммарную задержку при повторных оптимизациях маршрутов с теми же точками.

На уровне пользовательского интерфейса загрузка Яндекс Карт реализована через утилиту `loadYandexMaps()`, которая инжектирует `<script>`-тег динамически при первом обращении и возвращает один и тот же Promise при последующих вызовах (одиночный экземпляр). Такой подход предотвращает дублирующую загрузку скрипта при одновременном обращении нескольких компонентов и исключает состояние гонки при инициализации API.

3.2.2 Надёжность аутентификации

Аутентификация реализована по схеме JWT [?] с двумя токенами: краткосрочным `access`-токеном (время жизни — 15 минут) и долгосрочным `refresh`-токеном (время жизни — 30 дней). Токены обновления хранятся в базе данных в виде хэша `bcrypt` [?], что исключает их компрометацию при несанкционированном доступе к СУБД.

На клиентской стороне реализован механизм автоматического обновления токена при получении ответа с кодом 401. Для исключения множественных одновременных запросов на обновление применяется дедупликация: при первом истечении токена запускается единственный запрос обновления, остальные запросы ожидают его завершения через общий `Promise`. После успешного обновления исходный запрос повторяется автоматически — пользователь не замечает прерывания сессии.

Сессия пользователя сохраняется в `localStorage` под ключом `planner.auth.session`. При закрытии и повторном открытии браузера приложение автоматически восстанавливает состояние аутентификации без повторного ввода учётных данных.

3.2.3 Надёжность инфраструктуры

При развёртывании с использованием `Docker Compose` [6] зависимость между контейнерами управляется через механизм `healthcheck`: сервис приложения миграций и API-сервер запускаются только после того, как контейнер базы данных PostgreSQL подтвердит готовность к приёму соединений командой `pg_isready`. Это исключает ситуацию, при которой API-сервер стартует до завершения инициализации СУБД.

Схема базы данных управляется инструментом `migrate/migrate`, запускаемым автоматически при каждом старте стека. Применение миграций является идемпотентным: повторный запуск `migrate up` при уже ак-

туальной схеме не вносит изменений. Это позволяет безопасно перезапускать стек без ручного контроля состояния миграций.

3.2.4 Производительность

Производительность серверной части обеспечивается несколькими механизмами. Язык Go компилируется в нативный исполняемый файл, что устраняет накладные расходы интерпретируемых сред. Драйвер `pgx/v5` использует пул соединений к PostgreSQL, исключая создание нового соединения на каждый HTTP-запрос. HTTP-фреймворк Gin применяет `httprouter` с маршрутизацией на дереве-префикса (`radix tree`), обеспечивая постоянное время поиска маршрута вне зависимости от числа зарегистрированных эндпоинтов.

Вычисления матрицы расстояний при оптимизации маршрута выполняются на стороне клиента посредством Яндекс Карт API, что разгружает сервер от вычислительно ёмких операций и позволяет обслуживать большее число одновременных пользователей без изменения аппаратной конфигурации.

Требование нефункциональных характеристик об устойчивости к ограничениям внешних API выполнено за счёт кэша расстояний: при повторных оптимизациях маршрутов с теми же парами точек обращения к OSRM и Яндекс Картам не производятся.

3.2.5 Вывод

Применённые технические решения обеспечивают устойчивость приложения к отказам внешних зависимостей, надёжную аутентификацию с автоматическим обновлением сессии и производительность, достаточную для обслуживания целевой аудитории системы.

3.3 Внедрение и сопровождение программного продукта

3.3.1 Модель распространения

Разработанное приложение является веб-приложением с клиент-серверной архитектурой и не требует установки специализированного программного обеспечения на рабочее место конечного пользователя. Пользователь взаимодействует с системой через стандартный веб-браузер.

Серверная часть распространяется в виде исходного кода и `Dockerfile`, что позволяет развернуть приложение на любой платформе с поддержкой Docker (Linux, macOS, Windows). Клиентская часть собирается в набор статических файлов (HTML, CSS, JavaScript) командой `pnpm build` и публикуется на любом веб-сервере (Nginx, Apache, Caddy и др.) без дополнительных зависимостей времени исполнения.

3.3.2 Процедура первоначального развёртывания

Рекомендуемый способ первоначального развёртывания — использование Docker Compose, подробно описанное в разделе «Руководство администратора» второй главы. Данный способ автоматически: поднимает контейнер PostgreSQL с расширением PostGIS; дожидается готовности СУБД через `healthcheck`; применяет SQL-миграции через `migrate/migrate`; запускает API-сервер на порту 8080.

Для продуктивной эксплуатации перед запуском необходимо:

- а) Заменить пароли СУБД по умолчанию в файле `docker-compose.yml`.
- б) Задать переменную `APP_JWT_SECRET` — криптографически стойкую случайную строку длиной не менее 32 символов.
- в) Настроить значение `APP_CORS_ORIGINS`, указав реальный адрес клиентской части.

г) Получить API-ключ Яндекс Карт на портале `developer.tech.yandex.ru` и записать его в файл `.env.local` клиентской части:

```
VITE_YANDEX_GEOCODER_KEY=<ключ>
```

д) Разместить клиентскую сборку (`dist/`) на веб-сервере, настроив перенаправление всех запросов на `index.html` и проксирование `/api/` на серверную часть.

3.3.3 Сопровождение и обновление

Процедура обновления приложения до новой версии сводится к одной команде:

```
docker compose up --build
```

Docker Compose пересобирает образ API-сервера и автоматически применяет новые миграции схемы базы данных до запуска обновлённого сервера. Данные PostgreSQL сохраняются в именованном томе `db_data` и не затрагиваются при пересборке образов.

Резервное копирование данных выполняется утилитой `pg_dump`:

```
docker exec <container_db> pg_dump -U planner planner > bac
```

Мониторинг состояния контейнеров обеспечивается стандартными инструментами Docker: `docker compose ps` показывает статус всех сервисов, `docker compose logs api` выводит журнал API-сервера.

3.3.4 Ограничения и направления развития

В текущей реализации хранение географических координат задач в базе данных осуществляется в соответствии с условиями бесплатного тарифа Яндекс Карт, которые допускают это для учебных и некоммерческих проектов. Для продуктивного использования в коммерческой среде потребуется переход на коммерческую лицензию API.

Суточный лимит бесплатного тарифа составляет 1 000 запросов к геокодеру и 25 000 запросов к JavaScript API. При превышении указанных лимитов рекомендуется рассмотреть переход на коммерческий тариф либо интеграцию с альтернативным геокодером.

Среди перспективных направлений развития системы можно выделить:

- реализацию push-уведомлений о наступлении временного окна следующей задачи;
- добавление мобильной версии интерфейса с адаптивной вёрсткой;
- поддержку командной работы с общим пулом задач;
- интеграцию с внешними календарями (Google Calendar, Яндекс Календарь);
- расширение алгоритма оптимизации для учёта транспортного средства и реального дорожного трафика.

3.3.5 Вывод

Разработанный программный продукт готов к развёртыванию в продуктивной среде. Контейнеризация на основе Docker обеспечивает воспроизводимость установки, автоматическое управление схемой базы данных и независимость от конфигурации целевой платформы. Процедуры резервного копирования, мониторинга и обновления не требуют специализированных инструментов и выполняются стандартными средствами Docker.

ЗАКЛЮЧЕНИЕ

Описание проделанной работы

В результате работы были выполнены следующие задачи:

1. Задача 1
2. Задача 2

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Бондаренко Т. В., Гарибов А. И., Федотов Е. А. Решение задачи планирования маршрутов с учётом временных окон // ИВД. 2020. №5 (65). URL: <https://cyberleninka.ru/article/n/reshenie-zadachi-planirovaniya-marshrutov-s-uchyotom-vremennyh-okon> (дата обращения: 27.02.2026).
2. The Go Programming Language [Электронный ресурс]. — Режим доступа: <https://go.dev/>, свободный (дата обращения: 15.03.2026).
3. PostgreSQL: The World's Most Advanced Open Source Relational Database [Электронный ресурс]. — Режим доступа: <https://www.postgresql.org/docs/>, свободный (дата обращения: 15.03.2026).
4. PostgreSQL Documentation. GiST Indexes [Электронный ресурс]. — Режим доступа: <https://www.postgresql.org/docs/current/gist.html>, свободный (дата обращения: 15.03.2026).
5. React — The library for web and native user interfaces [Электронный ресурс]. — Режим доступа: <https://react.dev/>, свободный (дата обращения: 15.03.2026).
6. Docker: Accelerated Container Application Development [Электронный ресурс]. — Режим доступа: <https://www.docker.com/>, свободный (дата обращения: 15.03.2026).
7. Relog — сервис оптимизации маршрутов [Электронный ресурс]. — Режим доступа: <https://relog.ru/>, свободный (дата обращения: 20.03.2026).
8. Яндекс Маршрутизация — сервис автоматического построения маршрутов [Электронный ресурс]. — Режим доступа: <https://routing.yandex.ru/>, свободный (дата обращения: 20.03.2026).
9. Spoke — Route Optimization Software [Электронный ресурс]. — Режим доступа: <https://www.spokeapp.io/>, свободный (дата обращения: 20.03.2026).

ПРИЛОЖЕНИЕ А

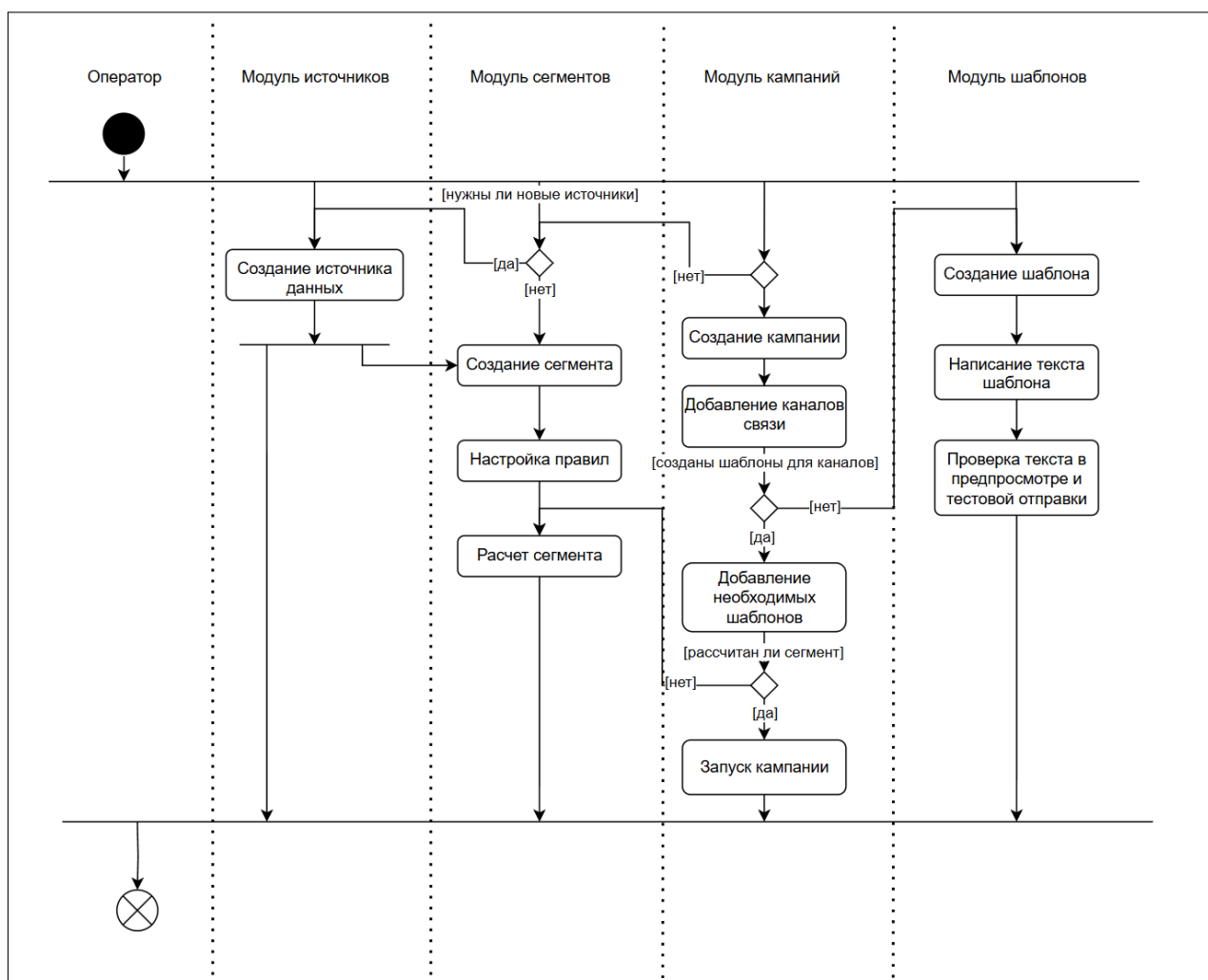


Рисунок А.1 – Создание кампании сегментированных рассылок

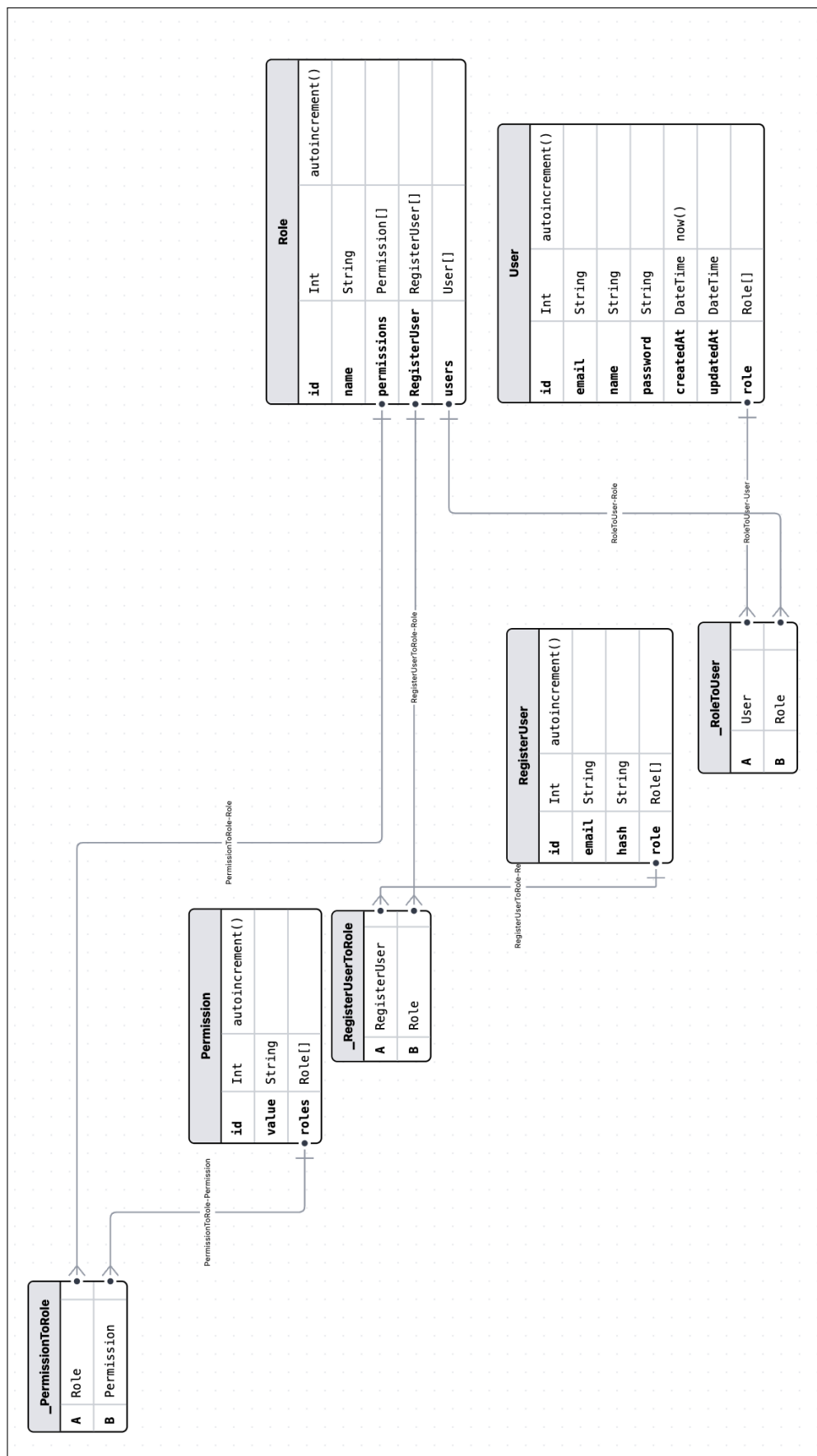


Рисунок А.2 – Даталогическая модель пользователя

Листинг 3.1 – Реализация «permissions.guard.ts»

```
1 @Injectable()
2 export class PermissionsGuard implements CanActivate {
3   constructor(
4     private reflector: Reflector,
5     private permissionsService: PermissionsService,
6   ) {}
7
7   canActivate(context: ExecutionContext): boolean {
8     try {
9       const requiredPermissions = this.reflector
10         .get<string[]>('permissions', context.getHandler())
11         .join('_');
12       if (!requiredPermissions) { return true }
13       const request = context.switchToHttp().getRequest();
14       const user = request.user;
15       return this.permissionsService.hasPermission(user,
16 ↪     requiredPermissions);
17     } catch {
18       return false;
19     }
20 }
```


Листинг 3.2 – Конфигурация для «docker-compose»

```
1 services:
2   documentation:
3     build: ./docs
4     ports:
5       - "3002:80"
6   app:
7     build: ./backand
8     ports:
9       - "3000:3000"
10    depends_on:
11      - db
12      - donorBb
13    env_file: ./backand/.env
14    environment:
15      - DATABASE_URL=postgresql://${DATABASE_USER}:${DATABASE_PASSWORD}@db:5432/${DATABASE_NAME}
16      - DONOR_DATABASE_URL=postgresql://${DONOR_DATABASE_USER}:${DONOR_DATABASE_PASSWORD}@donorBb:5432/${DONOR_DATABASE_NAME}
17      - DONOR_DATABASE_PORT=5432
18  frontend:
19    build: ./frontend
20    ports:
21      - "3001:3000"
22    depends_on:
23      - app
24    env_file: ./frontend/.env
25    environment:
26      - NEXT_PUBLIC_BASE_URL=/api
27  nginx:
28    image: nginx:latest
29    ports:
30      - "80:80"
31      - "443:443"
```

Продолжение листинга 3.2

```
1  volumes:
2    - ./nginx.conf:/etc/nginx/conf.d/default.conf:ro
3    - /etc/letsencrypt/:/etc/letsencrypt:ro
4  depends_on:
5    - frontend
6    - app
7    - documentation
8  db:
9    image: postgres:16
10   ports:
11     - "5432:5432"
12   env_file:  ./backand/.env
13   environment:
14     - POSTGRES_USER=${DATABASE_USER}
15     - POSTGRES_PASSWORD=${DATABASE_PASSWORD}
16     - POSTGRES_DB=${DATABASE_NAME}
17   volumes:
18     - db_data:/var/lib/postgresql/data
19  donorBb:
20    image: postgres:16
21    ports:
22      - "5433:5432"
23    env_file:  ./backand/.env
24    environment:
25      - POSTGRES_USER=${DONOR_DATABASE_USER}
26      - POSTGRES_PASSWORD=${DONOR_DATABASE_PASSWORD}
27      - POSTGRES_DB=${DONOR_DATABASE_NAME}
28    volumes:
29      - donor_data:/var/lib/postgresql/data
30  volumes:
31    db_data:
32    donor_data:
```


ПРИЛОЖЕНИЕ Б

Листинг 3.1 – Алгоритм ближайшего соседа с временными окнами
(nearest_neighbor.go)

```
1 package optimizer

2 import (
3     ^I"context"
4     ^I"math"
5 )

6 // NearestNeighborTW реализует эвристику ближайшего соседа
7   → для задачи
8 // коммивояжёра с временными окнами (NNH-TW).
9 // Алгоритм:
10 // 1. Выбрать начальный узел: с наиболее ранним открытием
11   → временного окна
12 //    или узел 0, если временные окна не заданы.
13 // 2. Отметить его посещённым; установить текущее время =
14   → startTime + длительность.
15 // 3. Повторять, пока не посещены все узлы:
16 //    а. Проход 1: найти ближайший допустимый узел
17   → (укладывается в окно).
18 //    б. Проход 2: если таких нет – взять ближайший без
19   → учёта окна.
20 //    в. Вычислить время ожидания, если прибываем до
21   → открытия окна.
22 //    г. Сдвинуть текущее время: прибытие + ожидание +
23   → длительность обслуживания.
24 // 4. Вернуть упорядоченные индексы узлов и агрегированную
25   → статистику.
26 //
27 // Временная сложность:  $O(n^2)$ .
28 type NearestNeighborTW struct{}

29 func NewNearestNeighborTW() *NearestNeighborTW { return
30   → &NearestNeighborTW{} } 52
```

Листинг 3.2 – HTTP-обработчики эндпоинтов маршрутов (handlers_routes.go)

```
1 package http

2 import (
3     ^I"net/http"

4     ^I"github.com/gin-gonic/gin"
5     ^Iroutestory "planner-backend/internal/domain/route/story"
6     ^I"planner-backend/internal/platform/distance"
7 )

8 type RouteHandlers struct {
9     ^Istory *routestory.Story
10 }

11 func NewRouteHandlers(st *routestory.Story) *RouteHandlers {
12     ^Ireturn &RouteHandlers{story: st}
13 }

14 func (h *RouteHandlers) Create(c *gin.Context) {
15     ^IuserID, ok := userIDFromCtx(c)
16     ^Iif !ok {
17         ^I^Ireturn
18     ^I}
19     ^Ivar req CreateRouteReq
20     ^Iif err := c.ShouldBindJSON(&req); err != nil {
21         ^I^Ic.JSON(http.StatusBadRequest, gin.H{"error": "bad
           ↳ request"})
22     ^I^Ireturn
23     ^I}
24     ^Iout, err := h.story.CreateDraft(
25     ^I^Ic.Request.Context(), userID, req.Source,
           ↳ req.OrderedTaskIDs,
26     ^I)
27     ^Iif err != nil {
28         ^I^Ic.JSON(http.StatusBadRequest, gin.H{"error":
           ↳ err.Error()})
29     ^I^Ireturn
```

Листинг 3.3 – Ключевые функции API-клиента: авторизованные запросы и обновление токена (client.ts)

```
1 // Единственный глобальный промис обновления токена –  
  → предотвращает  
2 // параллельные запросы /auth/refresh при одновременных  
  → 401-ошибках.  
3 let refreshSessionPromise: Promise<AuthSession> | null =  
  → null;  
  
4 // optimizeRoute передаёт идентификаторы задач и  
  → предварительно вычисленную  
5 // матрицу расстояний Яндекс.Карт на бэкенд.  
6 // Передача матрицы из buildYandexDistanceMatrix()  
  → обеспечивает согласованность  
7 // данных оптимизации с отображением на карте.  
8 export async function optimizeRoute(  
9   taskIds: string[],  
10  options: AuthorizedRequestOptions,  
11  startTimeMins = 540,  
12  distanceMatrix?: DistanceCell[][],  
13 ): Promise<SavedRouteDetails> {  
14   const route = await requestWithAuth<ApiRouteFull>(  
15     '/routes/optimize',  
16     {  
17       method: 'POST',  
18       body: JSON.stringify({ taskIds, startTimeMins,  
19         → distanceMatrix }),  
20     },  
21     options,  
22   );  
23   return mapRouteDetails(route);  
24 }  
  
24 // requestWithAuth выполняет HTTP-запрос с токеном доступа.  
25 // При получении 401 однократно обновляет токен и повторяет  
  → запрос.  
26 async function requestWithAuth<T>(  
27   path: string,  
28   init: RequestInit,
```

Листинг 3.4 – Построение матрицы расстояний через Яндекс Маршруты JS API (routeOptimizer.ts)

```
1 import { Task } from '../types/task';
2 import { loadYandexMaps } from '../yandexMaps';

3 export interface DistanceCell {
4   distanceM: number;
5   durationSec: number;
6 }

7 /**
8  * Строит матрицу расстояний n×n для списка задач через
9  * → Яндекс Маршруты.
10 * Используется тот же движок, что и для визуализации
11 * → маршрута на карте,
12 * поэтому данные оптимизации согласованы с отображением.
13 *
14 * matrix[i][j] = стоимость проезда от tasks[i] до tasks[j].
15 * Диагональные элементы (i === j) равны нулю.
16 */
17 export async function buildYandexDistanceMatrix(
18   tasks: Task[],
19   routingMode: 'auto' | 'pedestrian' | 'masstransit' =
20   → 'auto',
21 ): Promise<DistanceCell[][]> {
22   const n = tasks.length;
23   if (n === 0) return [];
24
25   await loadYandexMaps();
26
27   // masstransit через multiRouter не возвращает надёжные
28   → данные по времени —
29   // используем 'auto' как приближение.
30   const mode = routingMode === 'masstransit' ? 'auto' :
31   → routingMode;
32
33   const fallbackSec = 99_999;
34   const matrix: DistanceCell[][] = Array.from({ length: n },
35   → (_, i) =>
```